

Algorithmique et programmation pour l'ingénieur

TP3 — Évaluation des performances du vecteur

Introduction

Dans le TP précédent, vous avez développé en C une première implémentation d'un tableau dynamique inspiré de la classe C++ « **Vector** ». Cette structure permet de stocker des valeurs de type `double`, d'accéder à ces valeurs par indice, ainsi que d'ajouter ou de supprimer dynamiquement des éléments.

Dans ce TP, vous allez compléter cette bibliothèque avec des outils permettant d'évaluer expérimentalement ses performances. Vous étudierez notamment :

- le temps d'exécution de différentes opérations ;
- l'impact de la taille initiale du vecteur ;
- l'impact du nombre d'opérations réalisées ;
- le comportement de différentes versions de compilation ;
- la couverture de code des tests unitaires.

Ce TP introduit également une petite bibliothèque de génération aléatoire qui sera utilisée pour créer des scénarios de test variés.

1 Objectifs du TP

L'objectif principal de ce TP est de mesurer et comparer les performances de votre implémentation de tableau dynamique.

Vous devrez mettre en place plusieurs scénarios de benchmark :

- ajout et suppression d'éléments à des positions aléatoires ;
- ajout et suppression d'éléments en tête ;
- ajout et suppression d'éléments en queue ;
- lectures et écritures à des positions aléatoires ;
- lectures et écritures séquentielles ;
- tri de données par tri à bulles.

Complexité attendue des opérations sur un tableau dynamique

Pour un tableau dynamique simple :

- l'accès à un élément par indice est en temps constant : $O(1)$;
- l'écriture à un indice donné est en temps constant : $O(1)$;
- l'insertion ou la suppression en tête nécessite de décaler les éléments : $O(n)$;
- l'insertion ou la suppression à une position quelconque est en moyenne en $O(n)$;
- l'insertion ou la suppression en queue peut être plus rapide, mais dépend de la stratégie de gestion mémoire utilisée.

Les mesures expérimentales réalisées dans ce TP doivent permettre d'observer ces tendances.

Attention aux mesures expérimentales

Une mesure isolée n'est pas toujours représentative. Pour obtenir des résultats exploitables, il est conseillé de répéter les expériences, d'éviter les valeurs trop petites et de comparer les tendances générales plutôt que les valeurs exactes.

2 Réorganisation du projet

Dans l'archive du TP3, le projet est organisé avec plusieurs sous-répertoires :

- `./src` : répertoire contenant les fichiers source C du projet ;
- `./headers` : répertoire contenant les fichiers d'en-tête du projet ;
- `./objs` : répertoire où sont générés les fichiers objet ;
- `./bin` : répertoire où sont générés les exécutables ;
- `./coverage` : répertoire où sont générés les rapports de couverture de code.

Vous devez recopier les fichiers de votre TP précédent dans cette nouvelle organisation.

2.1. Créez ou utilisez le répertoire de travail « `utr_eapi_lib` ».

2.2. Rangez vos fichiers du TP précédent dans les bons sous-répertoires :

- `vector.c` dans `./src` ;
- `test_unit.c` dans `./src` ;
- `vector.h` dans `./headers` ;
- `README` à la racine du projet.

2.3. Vérifiez que les nouveaux fichiers fournis sont présents :

- `./src/random.c` ;
- `./headers/random.h` ;
- `./src/bench_vector.c` ;
- `./src/conditional_compilation_demo.c` ;
- `Makefile`.

Bonnes pratiques à respecter

- **Ajoutez des commentaires dans vos codes.**
- **Vérifiez les arguments passés aux fonctions** : pointeurs non NULL, indices valides, tailles non nulles lorsque c'est nécessaire, etc.
- **Utilisez des tests unitaires avec `ASSERT()`** pour vérifier le bon fonctionnement des fonctions.
- **Vérifiez régulièrement votre code avec `valgrind`.**

3 Compilation, versions et couverture de code

Le nouveau `Makefile` permet de compiler plusieurs exécutables et de produire différentes variantes du projet.

3.1 Compilation standard

Pour compiler le projet, utilisez :

```
make
```

Cette commande génère les exécutables suivants dans le répertoire `./bin` :

- `test_unit` ;
- `bench_vector` ;
- `conditional_compilation_demo`.

3.2 Compilation en mode debug

Pour compiler en mode debug, utilisez :

```
make DEBUG=1
```

Cette commande génère des exécutables suffixés par `_debug`.

3.3 Compilation avec différentes versions

Le `Makefile` permet de définir la constante `VERSION` lors de la compilation :

```
make VERSION=2  
make VERSION=3
```

Ces commandes permettent de produire différentes versions des exécutables.

Compilation conditionnelle

La compilation conditionnelle permet d'inclure ou d'exclure certaines parties du code au moment de la compilation. Elle repose sur des directives du préprocesseur telles que `#ifdef`, `#ifndef`, `#elif` et `#endif`.

Dans ce TP, la constante `VERSION` est utilisée pour illustrer ce mécanisme.

3.4 Couverture de code

Pour générer un rapport de couverture de code, utilisez :

```
make coverage
```

Le rapport HTML est généré dans le répertoire `./coverage`.

Couverture de code

La couverture de code permet de mesurer quelles parties du programme sont exécutées lors des tests unitaires. Elle aide à identifier les fonctions ou branches de code insuffisamment testées. Une couverture élevée ne garantit pas l'absence d'erreurs, mais elle constitue un bon indicateur de la qualité de la campagne de tests.

3.1. Ouvrez un terminal dans le répertoire du projet et exécutez les commandes suivantes :

(a) Compilez le projet avec :

```
make
```

(b) Exécutez les tests unitaires :

```
./bin/test_unit
```

(c) Compilez le projet en mode debug :

```
make DEBUG=1
```

(d) Exécutez la version debug des tests :

```
./bin/test_unit_debug
```

(e) Générez les informations de couverture de code :

```
make coverage
```

(f) Nettoyez les fichiers générés :

```
make clean
```

3.2. Utilisez la commande suivante pour afficher les variables principales utilisées par le `Makefile` :

```
make info
```

Expliquez le rôle des variables `SRCS`, `OBJS` et `TARGETS_BIN`.

4 Compilation conditionnelle

Le fichier `./src/conditional_compilation_demo.c` illustre l'utilisation de la compilation conditionnelle. Le comportement du programme dépend de la constante `VERSION` définie au moment de la compilation.

Dans ce TP, la compilation conditionnelle est uniquement présentée comme un mécanisme à comprendre. Elle ne sera pas utilisée directement pour modifier l'implémentation de votre vecteur.

Pourquoi introduire la compilation conditionnelle ?

Dans le TP suivant, nous utiliserons la compilation conditionnelle pour conserver plusieurs implémentations de la structure `vector` dans le même code source.

L'objectif sera de pouvoir compiler différentes versions de la bibliothèque, par exemple une version naïve et une version optimisée, sans créer plusieurs fichiers presque identiques contenant beaucoup de duplication de code.

Cela permettra ensuite de comparer les performances des différentes implémentations avec les mêmes programmes de test et de benchmark.

4.1. Compilez successivement les trois variantes suivantes :

(a) Version par défaut :

```
make
```

(b) Version 2 :

```
make VERSION=2
```

(c) Version 3 :

```
make VERSION=3
```

4.2. Exécutez les programmes générés :

```
./bin/conditional_compilation_demo  
./bin/conditional_compilation_demo_v2  
./bin/conditional_compilation_demo_v3
```

4.3. Observez les messages affichés. Expliquez pourquoi ils sont différents en vous appuyant sur le contenu du fichier `conditional_compilation_demo.c`.

4.4. Expliquez en quelques lignes pourquoi ce mécanisme pourra être utile pour comparer plusieurs implémentations de `vector` sans multiplier les fichiers source.

5 Générateur aléatoire

Dans cette partie, vous allez compléter une petite bibliothèque de génération aléatoire. Elle sera utilisée ensuite pour générer des indices, des valeurs et des chaînes de caractères lors des tests.

En C, la fonction standard `rand()` permet de produire des nombres pseudo-aléatoires. Cette fonction retourne un entier compris entre 0 et `RAND_MAX` inclus.

Nombres pseudo-aléatoires

La fonction `rand()` ne produit pas de vrais nombres aléatoires. Elle produit une suite déterministe de nombres qui dépend d'une graine initiale. Deux exécutions utilisant la même graine produisent donc la même suite de valeurs.

Initialisation avec `srand()`

La fonction `srand()` permet d'initialiser la graine du générateur pseudo-aléatoire. Pour obtenir une suite différente à chaque exécution, on utilise souvent l'heure courante :

```
1 srand(time(NULL));
```

Dans ce TP, cette initialisation est encapsulée dans la fonction `random_init()`.

Attention aux intervalles

Dans ce TP, les fonctions de génération aléatoire ne suivent pas toutes la même convention d'intervalle :

- `random_double(a, b)` retourne une valeur dans $[a, b[$;
- `random_float(a, b)` retourne une valeur dans $[a, b[$;
- `random_size_t(a, b)` retourne une valeur dans $[a, b]$;
- `random_int(a, b)` retourne une valeur dans $[a, b]$;
- `random_char(a, b)` retourne une valeur dans $[a, b]$.

Pensez à en tenir compte dans vos tests unitaires.

Biais de l'opérateur modulo

L'utilisation de `rand() % n` peut introduire un biais lorsque `RAND_MAX + 1` n'est pas divisible par `n`. Pour ce TP, on privilégiera une mise à l'échelle à partir de la valeur `rand() / (RAND_MAX + 1.0)`.

5.1. Complétez les fonctions du fichier `./src/random.c`.

(a) Complétez la fonction :

```
1 void random_init(uint32_t seed);
```

Elle doit initialiser le générateur pseudo-aléatoire à partir de la graine fournie.

(b) Complétez la fonction :

```
1 double random_double(double a, double b);
```

Elle doit retourner un nombre aléatoire de type `double` dans l'intervalle $[a, b[$. Si `a > b`, les bornes doivent être échangées.

(c) Complétez la fonction :

```
1 float random_float(float a, float b);
```

Elle doit retourner un nombre aléatoire de type `float` dans l'intervalle $[a, b[$. Si `a > b`, les bornes doivent être échangées.

(d) Complétez la fonction :

```
1 size_t random_size_t(size_t a, size_t b);
```

Elle doit retourner une valeur de type `size_t` dans l'intervalle $[a, b]$. Si `a > b`, les bornes doivent être échangées.

(e) Complétez la fonction :

```
1 int random_int(int a, int b);
```

Elle doit retourner une valeur de type `int` dans l'intervalle $[a, b]$. Si `a > b`, les bornes doivent être échangées.

(f) Complétez la fonction :

```
1 char random_char(char a, char b);
```

Elle doit retourner une valeur de type `char` dans l'intervalle $[a, b]$. Si $a > b$, les bornes doivent être échangées.

(g) Complétez la fonction :

```
1 void random_init_string(char *c, size_t n);
```

Elle doit remplir la chaîne `c` avec des lettres majuscules aléatoires entre 'A' et 'Z'. La fonction doit écrire $n - 1$ caractères puis terminer la chaîne avec `'\0'`.

5.2. Ajoutez des tests unitaires dans `./src/test_unit.c` pour vérifier les fonctions de `random.c`.

(a) Vérifiez que les valeurs générées sont bien dans les intervalles attendus.

(b) Vérifiez le comportement lorsque les bornes sont inversées.

(c) Vérifiez le comportement lorsque les deux bornes sont égales.

(d) Vérifiez que `random_init_string()` génère uniquement des majuscules et ajoute bien le caractère `'\0'`.

6 Benchmark du vecteur dynamique

Le fichier `./src/bench_vector.c` contient un programme permettant de lancer différents scénarios de test.

Le programme s'utilise avec trois arguments :

```
./bin/bench_vector <test_type> <init_size> <n>
```

avec :

- `test_type` : le scénario de benchmark à exécuter ;
- `init_size` : la taille initiale du vecteur ;
- `n` : le nombre d'opérations ou de répétitions à effectuer.

Les valeurs possibles pour `test_type` sont :

- `insert_erase_random` ;
- `insert_erase_head` ;
- `insert_erase_tail` ;
- `read_write_random` ;
- `read_write_sequential` ;
- `bubble_sort`.

Rôle du paramètre `n`

Le paramètre `n` représente le nombre d'opérations effectuées pendant le benchmark. Il permet d'obtenir un temps de mesure suffisamment grand pour être exploitable. Pour des opérations très rapides, il est nécessaire d'augmenter fortement `n`.

6.1. Complétez les fonctions du fichier `./src/bench_vector.c`.

(a) Complétez la fonction :

```
1 void insert_erase_random(size_t init_size, size_t n);
```

Cette fonction doit allouer un vecteur de taille `init_size`, puis répéter `n` fois une insertion et une suppression à une position aléatoire.

(b) Complétez la fonction :

```
1 void insert_erase_head(size_t init_size, size_t n);
```

Cette fonction doit allouer un vecteur de taille `init_size`, puis répéter `n` fois une insertion et une suppression en tête.

(c) Complétez la fonction :

```
1 void insert_erase_tail(size_t init_size, size_t n);
```

Cette fonction doit allouer un vecteur de taille `init_size`, puis répéter `n` fois une insertion et une suppression en queue.

(d) Complétez la fonction :

```
1 void read_write_random(size_t init_size, size_t n);
```

Cette fonction doit allouer un vecteur de taille `init_size`, puis répéter `n` fois une lecture et une écriture à une position aléatoire.

(e) Complétez la fonction :

```
1 void read_write_sequential(size_t init_size, size_t n);
```

Cette fonction doit allouer un vecteur de taille `init_size`, puis répéter `n` fois une lecture et une écriture avec un parcours séquentiel.

(f) Complétez la fonction :

```
1 void bubble_sort(size_t init_size, size_t n);
```

Cette fonction doit allouer un vecteur de taille `init_size`, puis répéter `n` fois :
— l'initialisation du vecteur avec des valeurs aléatoires;
— le tri du vecteur avec l'algorithme du tri à bulles.

6.2. Testez chaque scénario avec une petite valeur de `init_size` et une petite valeur de `n` afin de vérifier que le programme s'exécute correctement.

```
./bin/bench_vector insert_erase_random 10 100
./bin/bench_vector insert_erase_head 10 100
./bin/bench_vector insert_erase_tail 10 100
./bin/bench_vector read_write_random 10 100
./bin/bench_vector read_write_sequential 10 100
./bin/bench_vector bubble_sort 10 10
```

7 Mesures expérimentales

Choix de `init_size` et `n` et estimation du temps unitaire

Le couple de paramètres `init_size` (taille du tableau) et `n` (nombre d'opérations) doit être choisi de manière à obtenir des mesures de temps fiables.

Pour une taille donnée `init_size`, l'objectif est de choisir une valeur de `n` suffisamment grande pour que le temps total d'exécution soit de l'ordre de quelques secondes (par exemple entre 1 et 10 secondes). Cela permet de réduire l'impact du bruit de mesure (ordonnanceur, autres processus, etc.).

Une fois le temps total mesuré, on peut estimer le temps unitaire d'une opération en divisant ce temps par `n` :

$$\text{temps unitaire} \approx \frac{\text{tempstotal}}{n}$$

En répétant cette démarche pour différentes valeurs de `init_size`, on obtient une estimation de l'évolution du coût d'une opération en fonction de la taille du tableau.

Une fois les fonctions de benchmark complétées, vous devez mesurer les performances de votre structure.

7.1 Mesure du temps d'exécution

La commande `time` permet de mesurer le temps d'exécution d'un programme. Par exemple :

```
time ./bin/bench_vector insert_erase_random 1000 10000
```

Pour obtenir plus de détails, vous pouvez utiliser :

```
/usr/bin/time -v ./bin/bench_vector insert_erase_random 1000 10000
```

Valeurs retournées par time

La commande `time` affiche généralement plusieurs informations :

- **real** : temps réel écoulé entre le lancement et la fin du programme ;
- **user** : temps CPU utilisé par le programme en mode utilisateur ;
- **sys** : temps CPU utilisé par le programme en mode noyau.

Dans ce TP, vous pouvez principalement vous appuyer sur le temps **real** ou **user**, à condition de rester cohérent dans toutes vos mesures.

7.2 Mesure mémoire avec Valgrind

Pour détecter les erreurs mémoire et les fuites mémoire, utilisez :

```
valgrind --tool=memcheck --leak-check=full ./bin/bench_vector  
insert_erase_random 1000 10000
```

Pour analyser l'utilisation mémoire au cours de l'exécution, vous pouvez utiliser `massif` :

```
valgrind --tool=massif ./bin/bench_vector insert_erase_random 1000  
10000
```

Mesures et interprétation

Les mesures de performance dépendent fortement de la machine utilisée, de la charge du système et des options de compilation. Il est donc préférable de comparer des expériences réalisées sur la même machine, avec les mêmes options de compilation.

7.1. Réalisez des mesures de temps pour les scénarios suivants :

- `insert_erase_random` ;
- `insert_erase_head` ;
- `insert_erase_tail` ;
- `read_write_random` ;
- `read_write_sequential` ;
- `bubble_sort`.

7.2. Choisissez plusieurs valeurs de `init_size`, par exemple :

- 100
- 1000
- 10000
- ...

7.3. Choisissez plusieurs valeurs de `n`, en les adaptant au coût du scénario testé. Par exemple, `bubble_sort` étant coûteux, il faudra utiliser des valeurs plus faibles que pour les lectures/écritures.

- 7.4. Enregistrez vos résultats dans un tableau.
- 7.5. Produisez au moins deux graphiques :
 - (a) un graphique comparant les temps d'exécution des différentes opérations ;
 - (b) un graphique montrant l'évolution du temps d'exécution en fonction de `init_size` pour un scénario donné.

8 Analyse des résultats

Les mesures expérimentales doivent être interprétées en lien avec la complexité attendue des opérations.

- 8.1. Comparez les performances de `insert_erase_head`, `insert_erase_tail` et `insert_erase_random`. Que constatez-vous ?
- 8.2. Comparez les performances de `read_write_random` et `read_write_sequential`. Les résultats sont-ils proches ? Pourquoi ?
- 8.3. Analysez le coût du tri à bulles. Pourquoi ce scénario devient-il rapidement coûteux lorsque `init_size` augmente ?
- 8.4. Comparez les résultats obtenus avec la compilation standard et la compilation en mode debug. Observez-vous une différence ?